Vocals First: Source-Separated Speech as the Backbone of Story-Level Movie QA

Team: ZFTurbo

Roman Solovyev, MVSep, roman.solovyev.zf@gmail.com

SLoMO 2026 MovieQA Track (ECCV 2026 Workshop) — Main and Special Tracks

Code: https://github.com/ZFTurbo/SLoMO-QA-Competition-2026

Abstract

This report describes a submission to the SLoMO MovieQA track, which asks open-ended, story-level questions about full amateur short films. The system is *training-free*: it pairs an explicit audio-understanding front-end with an off-the-shelf video large language model. For every film the audio is demuxed, the voice stem is isolated with a BS-RoFormer music-source-separation model, and the vocals are transcribed with timestamps using Parakeet-TDT-0.6B-v3. The time-aligned transcript is injected into the prompt alongside 200 uniformly sampled frames, and a single video LLM answers each question in one pass. Qwen3.8-27B-FP8 in reasoning mode is submitted to the Main track (58.36 accuracy / 2.72 score on the public leaderboard) and Qwen3-VL-8B-Instruct to the Special (<10B activated) track (48.70 / 2.55). No component is fine-tuned.

1 Task and Motivation

The MovieQA track requires free-form answers to questions about 5–40 minute amateur short films from SF20K [1]. Unlike short-clip video QA, the questions probe motivations, causal chains, off-screen implications and character development, so an answer must draw on the whole narrative. Submissions are scored by an LLM judge (GPT-4.1-nano) that returns a binary correctness verdict (accuracy, 0–100) and a graded match quality in $[0, 5]$.

Two observations shaped the design. **(i) Dialogue carries most of the story.** In short films, motivations, names and causal links are usually spoken rather than shown; a purely visual model has to infer them from priors, which the benchmark explicitly penalises. **(ii) Raw film audio is hard for ASR.** Short films are heavily mixed with music, ambience and effects, and off-the-shelf ASR degrades sharply on such mixtures. Speech extraction is therefore treated as a first-class pipeline stage, and the vocal stem is separated before transcription.

The rest of the system is deliberately minimal: one video LLM call per question, no retrieval, no agentic loop, no fine-tuning. This makes the method easy to reproduce and the Special-track variant a drop-in model swap.

2 Method

Figure 1 summarises the pipeline. It has an offline preprocessing stage (run once per film) and an inference stage (run once per question).

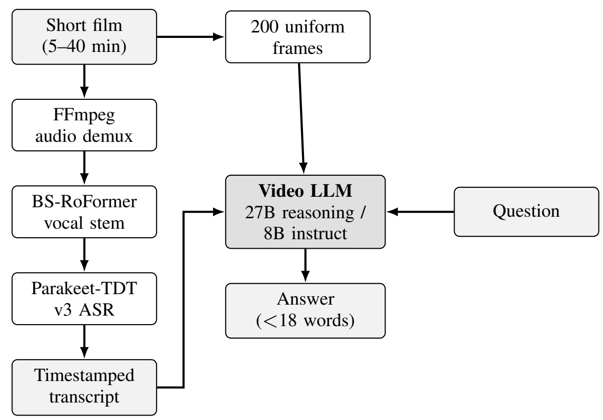

Figure 1: System overview. The audio branch (left) produces a time-aligned transcript; the video branch supplies 200 uniformly sampled frames. Both are concatenated with the question in a single prompt.

2.1 Audio branch

Demuxing. The audio track is extracted with FFmpeg to 44.1 kHz stereo 16-bit PCM WAV.

Voice separation. The BS-RoFormer 6-stem checkpoint[1] is run through the Music-Source-Separation-Training (MSST) framework [2, 3, 8] and the *vocals* stem is kept. The instrumental residual is also reconstructed as mix − vocals; it is unused by the current submission but is a natural hook for future music/ambience conditioning. The vocal stem is downmixed to 16 kHz mono, the rate expected by the ASR model.

Transcription. Transcription uses NVIDIA Parakeet-TDT-0.6B-v3 [4], a multilingual FastConformer-TDT model with native word- and segment-level timestamps. Films are split into 300-second chunks and per-chunk timestamps are shifted by the chunk offset, so they remain absolute with respect to the film. Of the three exported views (plain, word-level, segment-level), the *segment-level* file, formatted as `[start s - end s] : text`, is the one fed to the LLM: it gives the model an explicit temporal index it can align with the sampled frames.

2.2 Video branch

Rather than fixing a frame rate, the pipeline fixes a frame *budget*. For a film of duration T seconds and native rate f_0, frames

[1] `bs_6stem_fixed.ckpt` from the `mvsepless_resources` release.

are sampled at

$$f = \min\left(\frac{N}{T}, f_0\right), \qquad N = 200,$$

so every film contributes at most $N = 200$ frames regardless of length. This keeps the prompt inside the 131k-token context window and makes cost per film roughly constant across the 5–40 minute range, while covering the whole timeline instead of over-sampling an arbitrary window. Frames are resized within `min_pixels`$= 4\times32\times32$ and `max_pixels`$= 360\times420$; the decoding parameters are passed to the server both in the `video_url` field and in `mm_processor_kwargs` so that the frame budget is honoured end-to-end. Videos are read from local disk via `file://` URLs (`--allowed-local-media-path`), which avoids re-encoding and base64 transport.

2.3 Prompt

A single user turn contains the instruction, the full transcript and the question, followed by the video:

> *Based on the video and its transcript, give short and concise answer for the question. The timestamps in the text correspond to the time in the video. Answer must be less than 18 words.*
> *Video transcript:* {`transcript`}
> *Question:* {`question`}

Two details matter for the metric. First, the prompt states explicitly that transcript timestamps are aligned with the video, which encourages the model to use the transcript as a temporal index into the frames rather than as an independent text document. Second, the 18-word cap is a direct response to the judging protocol: verbose answers dilute the meaningful match against a short reference answer and empirically lower both accuracy and the graded score, whereas a terse, committed answer is judged more favourably.

2.4 Models and serving

All models are served with vLLM's[7] OpenAI-compatible server on a single GPU with `--max-model-len 131072` and `--gpu-memory-utilization 0.95`.

Main track. Qwen3.8-27B-FP8 [5] in reasoning mode, with `enable_thinking` and `preserve_thinking` on and `reasoning_effort="xhigh"`. Sampling uses temperature 1.0, `top_p`$=0.95$, and a 32,768-token budget to leave room for a long chain of thought. The final answer is recovered by splitting the completion on the </think> delimiter and keeping the trailing segment. Qwen3-VL-32B-Instruct-FP8 was also evaluated; it reaches a slightly lower score at substantially higher throughput.

Special track. Qwen3-VL-8B-Instruct [6], a dense 8B model, so **8B parameters are activated per token**, well inside the 10B limit. It runs in non-thinking mode with default sampling and a 128-token cap, which is sufficient given the 18-word answer constraint and makes the track variant far cheaper than the Main-track model. The pipeline is otherwise byte-identical: same preprocessing, same frame budget, same prompt.

Caching. Every (video, question) completion is written to disk before post-processing. On restart, cached Main-track completions are reused only if they contain a closing </think> tag, guarding against truncated reasoning traces being silently promoted to final answers. Long runs are thus restartable without recomputation, and a mid-run crash cannot corrupt the submission.

Model	Track	Acc.	Score
Qwen3.8-27B-FP8 (reasoning)	Main	**58.36**	**2.72**
Qwen3-VL-32B-Instruct-FP8	Main	∼52.0	∼2.60
Qwen3-VL-8B-Instruct	Special	48.70	2.55
Reference: public LB top-1	—	*66.36*	*3.02*

Table 1: SF20K-Test-Expert public split. Accuracy is the percentage of answers judged correct; score is the mean $[0, 5]$ quality rating.

3 Experiments

Hardware. All runs use a single NVIDIA A6000 96GB GPU (`tensor-parallel-size 1`). Preprocessing (separation + ASR) and inference are separate processes; the vLLM server is started once and driven over HTTP.

Results. Table 1 reports the public-test leaderboard.

Observations. (i) *Reasoning mode is worth its cost.* The 27B model in reasoning mode beats the larger 32B instruct model, at the price of a large drop in throughput — the practical trade-off for teams under a compute budget. (ii) *Small models are competitive.* The 8B Special-track model recovers roughly 83% of the Main-track accuracy at a fraction of the cost. This is attributable mainly to the transcript: once dialogue is supplied as clean, timestamped text, much of the story-level reasoning becomes text-grounded, and the visual stream is needed chiefly for grounding and disambiguation. (iii) *Separation before ASR matters.* Transcribing the raw mix gave visibly noisier transcripts on music-heavy films, with dropped and hallucinated lines; isolating the vocal stem first was the single most useful preprocessing decision.

4 Compliance and Reproducibility

No model is fine-tuned; all checkpoints are used out of the box, so no training data of any kind is involved and the test videos are used only for inference. No human labelling was performed on the test videos, and no external knowledge about specific test films was used. The full pipeline is three scripts — `preproc_data.py`, `run_main_track_...py` and `run_special_track_...py` — released with pinned requirements and the exact vLLM launch commands.

5 Limitations and Future Work

Captioning the unused instrumental stem would add score and ambience, both strong narrative cues. Speaker diarisation would let the transcript attribute lines to named characters, targeting the benchmark's character-grounding axis. Finally, a question-conditioned or shot-boundary-aware sampler should recover detail in event-dense passages without enlarging the prompt.

References

[1] R. Ghermi, X. Wang, V. Kalogeiton, I. Laptev. Short Film Dataset (SF20K): A Benchmark for Story-Level Video Understanding. *arXiv:2406.10221*, 2024.

[2] ZFTurbo. Music-Source-Separation-Training. `https://github.com/ZFTurbo/Music-Source-Separation-Training/`

[3] W.-T. Lu et al. Music Source Separation with Band-Split RoPE Transformer. *ICASSP*, 2024.

[4] NVIDIA NeMo. `parakeet-tdt-0.6b-v3`. `https://huggingface.co/nvidia/parakeet-tdt-0.6b-v3`

[5] Qwen Team. Qwen3 Technical Report. *arXiv:2505.09388*, 2025.

[6] Qwen Team. Qwen3-VL. `https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct`

[7] W. Kwon et al. Efficient Memory Management for LLM Serving with PagedAttention. *SOSP*, 2023.

[8] R. Solovyev, I. Kiselev, A. Stempkovskiy, T. Gabruseva. Music-Source-Separation-Training (MSST): A Unified Framework for Training and Evaluating Music Demixing Models. *arXiv:2607.23395*, 2026.